UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

FACULTAD DE INGENIERÍA EN SISTEMAS

CURSO: SISTEMAS OPERATIVOS

SECCIÓN "A"

CATEDRÁTICO: ING. EDWIN CÓRDOVA

MANUAL TÉCNICO DEL SIMULADOR DE PLANIFICADOR DE PROCESOS DEL PROYECTO FINAL DE SISTEMAS OPERATIVOS

NOMBRE:	CARNÉ:
DANIEL ANDRÉ SANDOVAL CHÁVEZ	0908-24-5040
DUSTIN STEVE SANTOS MONTERROSO	0908-24-3397
ERICK NEFTALÍ CAMARGO JIMÉNEZ	0908-23-9943
LUCIA VALERIA MOLLINEDO HERNÁNDEZ	0908-24-879
ALESSANDRA MARINELI CONTRERAS GÓMEZ	0908-24-5044

ESCUINTLA, OCTUBRE 2026

Índice

Manual Técnico - Simulador de Planificador de Procesos 1
1) Visión general 1
2) Arquitectura 2
3) Modelo de datos 3
4) Contrato de los planificadores 4
5) Algoritmos 4
FCFS (no apropiativo) 4
SJF, SRTF y Prioridades (`_por_clave`) 5
Round Robin 5
Traza de SRTF 6
6) Cálculo de métricas 6
7) Interfaz gráfica y animación del Gantt 7
8) Decisiones de diseño y casos borde 8
9) Pruebas 10
10) Complejidad, limitaciones y mejoras 11

Manual Técnico - Simulador de Planificador de Procesos

15 de octubre de 2026 | Hecho por: Daniel André Sandoval Chávez

1) Visión general

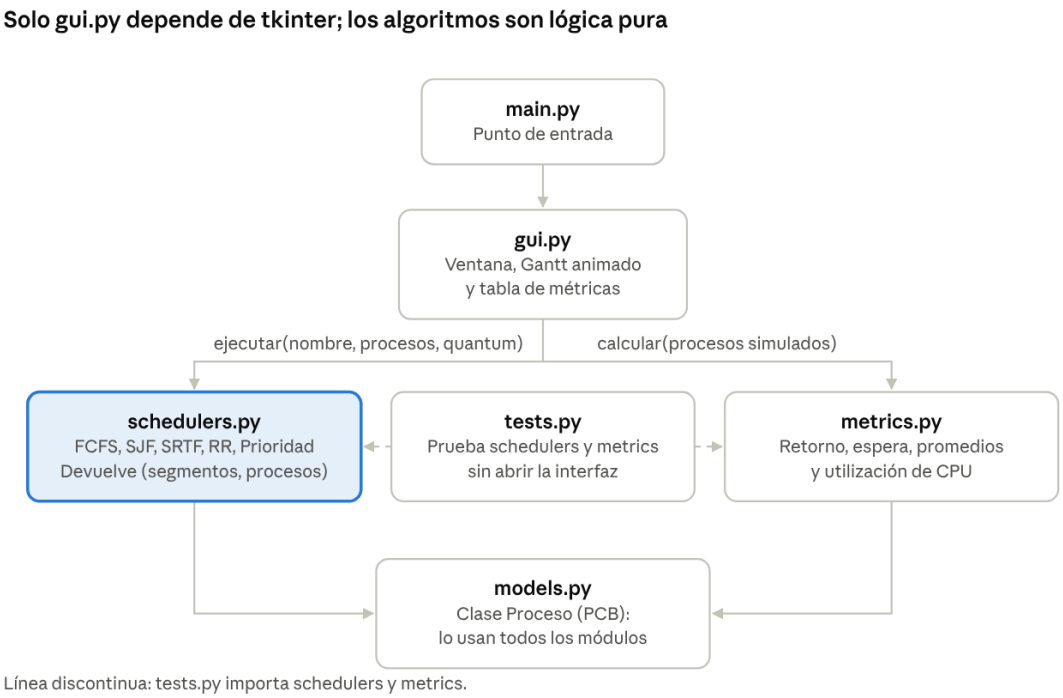
El simulador es una aplicación de escritorio en Python 3.9+ que ejecuta una misma lista de procesos con seis algoritmos de planificación de CPU y muestra el diagrama de Gantt animado y las métricas de cada ejecución. Usa solo la librería estándar: `tkinter` para la interfaz y `dataclasses` y `collections.deque` para la lógica.

Archivo	Responsabilidad
`models.py`	Clase `Proceso` (PCB simplificado).
`schedulers.py`	Algoritmos: FCFS, SJF/SRTF, Round Robin y Prioridades.
`metrics.py`	Cálculo de retorno, espera, promedios y utilización.
`gui.py`	Interfaz, animación del Gantt y tabla de métricas.
`main.py`	Punto de entrada.
`tests.py`	Pruebas con ejemplos resueltos a mano.

Algoritmos implementados: FCFS, SJF no apropiativo, SRTF (SJF apropiativo), Round Robin con quantum manual, y Prioridades apropiativo y no apropiativo como extra.

2) Arquitectura

Solo `gui.py` depende de `tkinter`: los algoritmos y las métricas son funciones puras que se prueban sin interfaz.



arquitectura de módulos = 6 archivos

`main.py` crea la ventana `App`. Al simular, `gui.py` llama a `schedulers.ejecutar()` y luego a `metrics.calcular();` ambos módulos trabajan con objetos `Proceso` de `models.py`. `schedulers.py` (resaltado) concentra la lógica de planificación. `tests.py` verifica los dos módulos sin abrir ninguna ventana.

3) Modelo de datos

Un proceso se representa con la `dataclass Proceso` de `models.py`, un bloque de control de proceso (PCB) simplificado con cinco atributos.

Atributo	Tipo	Origen	Descripción
`pid`	str	Usuario	Identificador del proceso.
`llegada`	int	Usuario	Tiempo de arribo (Arrival Time).
`rafaga`	int	Usuario	Tiempo de CPU requerido (Burst Time), mayor que 0.
`prioridad`	int	Usuario	Opcional; a menor número, mayor prioridad (por defecto 0).
`restante`	int	Simulación	Tiempo de CPU que aún le falta; inicia igual a `rafaga`.
`finalizacion`	int	Simulación	Instante en que termina (Completion Time).

`restante` se inicializa en `__post_init__`. El método `copia()` devuelve un proceso nuevo con solo los datos de entrada y la simulación en cero. Cada algoritmo trabaja sobre copias, así la misma lista original se ejecuta con todos los algoritmos sin que uno afecte al otro.

4) Contrato de los planificadores

Todos los algoritmos de `schedulers.py` reciben la lista original de procesos y devuelven la misma estructura, lo que permite a la interfaz y a las métricas tratarlos por igual.

```
segmentos, procesos = fcfs(lista) # también sjf(), round_robin(), prioridad()

segmentos # [("P1", 0, 8), ("P2", 8, 12), ("IDLE", 12, 15), ...]

procesos  # copias con `finalizacion` calculada, en el orden original
```

Elemento	Significado
Segmento `(pid, inicio, fin)`	Intervalo `[inicio, fin)` en que `pid` usa la CPU. La lista está en orden cronológico y es la base del Gantt.
`IDLE`	`Pid` reservado para los intervalos en que no hay procesos listos.
`procesos`	Copias de los procesos de entrada con `finalizacion` asignada; las métricas se calculan a partir de ellas.

Funciones de apoyo:

- `_fusionar(segmentos)` une segmentos contiguos del mismo proceso: `(P1,0,1)` y `(P1,1,3)` pasan a `(P1,0,3)`. Así el Gantt no muestra barras partidas.
- `ejecutar(nombre, procesos, quantum)` despacha por el nombre del algoritmo (lista `ALGORITMOS`); es la única función que llama la interfaz.

5) Algoritmos

Cuatro funciones cubren los seis algoritmos: FCFS y Round Robin tienen implementación propia, y SJF, SRTF y los dos de Prioridad comparten una sola función genérica.

FCFS (no apropiativo)

Ordena por llegada (`sorted` es estable, así los empates respetan el orden de ingreso) y ejecuta cada proceso completo.

```
t = 0
para cada proceso p ordenado por llegada:
    si t < p.llegada: registrar IDLE(t, p.llegada); t = p.llegada
    registrar (p.pid, t, t + p.rafaga)
    t = t + p.rafaga; p.finalizacion = t
```

SJF, SRTF y Prioridades (`_por_clave`)

Selecciona entre los procesos listos el de menor `clave`: el tiempo restante en SJF/SRTF o la prioridad en Prioridades. El desempate es la menor llegada. `sjf(lista, apropiativo)` y `prioridad(lista, apropiativo)` solo eligen la clave y el modo.

```
mientras queden procesos pendientes:
    listos = {p : p.restante > 0 y p.llegada <= t}
    si listos está vacío: registrar IDLE hasta la próxima llegada; continuar
    elegido = min(listos, por (clave, llegada))
    si no apropiativo: duracion = elegido.restante
    si apropiativo:    duracion = min(elegido.restante, proxima_llegada - t)
    registrar (elegido.pid, t, t + duracion)
    t += duracion; elegido.restante -= duracion
    si elegido.restante == 0: elegido.finalizacion = t
```

En modo apropiativo solo se reevalúa la decisión en cada llegada: entre llegadas, el proceso en ejecución solo mejora su clave y los demás no cambian, así que sigue siendo el mejor. Esto evita revisar en cada unidad de tiempo sin cambiar el resultado.

Round Robin

Usa dos colas: `por_llegar` (procesos que aún no llegan, ordenados) y `cola` (listos). La función interna `admitir(hasta)` mueve a `cola` los procesos con `llegada <= hasta`.

```
mientras queden procesos sin terminar:
    admitir(t)
    si cola está vacía: registrar IDLE hasta la próxima llegada; continuar
    p = cola.sacar_primero()
    duracion = min(quantum, p.restante)
    registrar (p.pid, t, t + duracion)
    t += duracion; p.restante -= duracion
    admitir(t)         # los que llegaron durante la ejecución entran primero
    si p.restante > 0: cola.agregar(p)
    si no: p.finalizacion = t
```

Si `quantum <= 0` lanza `ValueError`.

Traza de SRTF

Procesos P1(0,8), P2(1,4), P3(2,9), P4(3,5):

Instante	Listos (restante)	Decisión	Segmento
0	P1(8)	P1; próxima llegada en 1	(P1, 0, 1)
1	P1(7), P2(4)	P2 es más corto; corre hasta t = 5	(P2, 1, 5)
5	P1(7), P3(9), P4(5)	P4	(P4, 5, 10)
10	P1(7), P3(9)	P1	(P1, 10, 17)
17	P3(9)	P3	(P3, 17, 26)

P2 corre en tres tramos (1-2, 2-3, 3-5) que `_fusionar` une en uno solo.

6) Cálculo de métricas

`Metrics.calcular(procesos)` recibe los procesos ya simulados y calcula las mismas métricas para cualquier algoritmo.

```
T_{retorno} = T_{finalización} - T_{llegada}
T_{espera} = T_{retorno} - T_{ráfaga}
U_{CPU} = (Σ T_{ráfaga}/T_{final}) * 100
```

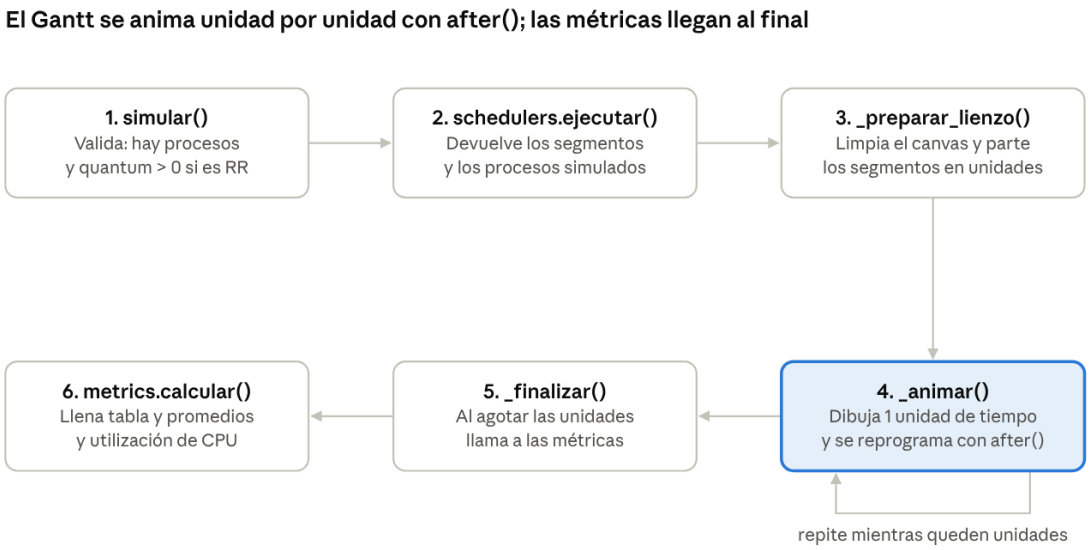
La función devuelve un diccionario:

Clave	Contenido
`filas`	Una por proceso: pid, llegada, ráfaga, fin, retorno, espera y los textos con el cálculo desarrollado (`f_retorno`, `f_espera`).
`prom_espera`, `prom_retorno`	Promedios globales (float).
`f_prom_espera`, `f_prom_retorno`	Texto con la suma desarrollada, por ejemplo `(0 + 7 + 10 + 18) / 4 = 8.75`.
`utilizacion`	Porcentaje del tiempo total en que la CPU estuvo ocupada.

Los textos desarrollados (`17 - 0 = 17`) permiten que la interfaz muestre el cálculo y no solo el resultado, como exige la rúbrica. El tiempo total `T_final` es la mayor `finalizacion`, de modo que los intervalos IDLE bajan la utilización por debajo de 100 %.

7) Interfaz gráfica y animación del Gantt

`gui.py` define la clase `App(tk.Tk)`, que organiza la ventana en cuatro paneles (procesos, algoritmo, Gantt y métricas) y guarda el estado de la simulación. El botón "Simular" ejecuta este flujo:



flujo de simular() = 6 pasos con un bucle

Estado de App:

Atributo	Uso
`procesos`	Lista de `procesos` ingresados por el usuario.
`_resultado`	Tupla `(segmentos, procesos simulados, algoritmo)` de la última simulación.
`_unidades`	Una tupla por unidad de tiempo a dibujar: `(pid, t, inicio_segmento, fin_segmento, es_ultimo).`
`_k`	Índice de la próxima unidad por dibujar.
`_job`	Identificador del `after()` pendiente; permite cancelar la animación.
`_colores`	Color asignado a cada PID (paleta de 10 colores).

Animación. `_animar()` dibuja la unidad `_k`, incrementa el contador y se vuelve a programar con `after(retardo)`. `_dibujar_unidad()` pinta el rectángulo de esa unidad; al abrir un segmento escribe su tiempo inicial y al cerrarlo dibuja el borde y la etiqueta del PID. Cada unidad mide 34 píxeles y el canvas se desplaza para mantener visible lo que se dibuja.

Control. `saltar()` cancela el `after()` pendiente, dibuja de golpe las unidades que faltan y llama a `_finalizar()`. Al iniciar una simulación nueva, `limpiar()` o `simular()` cancelan cualquier animación en curso con `_cancelar_animacion()`.

Comparación. `comparar()` ejecuta los seis algoritmos con la misma lista y muestra sus promedios en una ventana `Toplevel`.

Validaciones de entrada (`agregar()`): enteros válidos, llegada ≥ 0, ráfaga > 0 y PID único y distinto de IDLE. El quantum se valida en `_leer_quantum()`.

8) Decisiones de diseño y casos borde

Estas son las decisiones que conviene tener claras para la defensa, cada una con su motivo.

Decisión	Motivo
Los algoritmos trabajan sobre copias (`Proceso.copia()`).	La misma lista se ejecuta con todos los algoritmos sin que uno contamine al siguiente, y la lista original nunca se modifica.
Cada algoritmo devuelve segmentos `(pid, inicio, fin)`.	Una única representación alimenta el Gantt y las métricas; agregar un algoritmo nuevo no exige tocar la interfaz.
La lógica no importa `tkinter`.	Los algoritmos se prueban de forma automática y la interfaz se puede reemplazar sin reescribirlos.
SRTF reevalúa solo en las llegadas.	Entre llegadas el proceso en ejecución solo mejora su clave, por lo que no puede perder la CPU; el resultado es igual al de reevaluar en cada unidad, con menos pasos.
Un mismo `_por_clave` sirve a SJF, SRTF y Prioridades.	Los tres solo difieren en la clave de orden y en si son apropiativos; evita código duplicado.
Las métricas guardan el cálculo desarrollado como texto.	La rúbrica pide el cálculo expuesto en pantalla, no solo el resultado.

Casos borde y su tratamiento:

Caso	Tratamiento
Ningún proceso ha llegado (CPU libre)	Se registra un segmento `IDLE` hasta la próxima llegada, en todos los algoritmos.
Empate en SJF, SRTF o Prioridad	Gana la menor llegada; si persiste, el orden de ingreso (la función `min` devuelve el primero).
Llegada simultánea con fin de quantum (Round Robin)	El proceso que llega entra a la cola antes que el interrumpido, porque `admitir()` se ejecuta antes de reinsertarlo.
Quantum menor o igual a 0	`round_robin` lanza `ValueError`; la interfaz lo valida antes de llamarla.
Ráfaga igual a 0, llegada negativa o PID repetido	La interfaz rechaza el ingreso con un mensaje de error.
Procesos ingresados fuera de orden de llegada	Cada algoritmo los ordena internamente; el orden de ingreso solo sirve de desempate.

9) Pruebas

`tests.py` contiene 10 pruebas que comparan la salida de los algoritmos con resultados calculados a mano, la mayoría con el ejemplo clásico P1(0,8), P2(1,4), P3(2,9), P4(3,5). No usan librerías externas ni abren la interfaz.

```
python tests.py
```

La salida esperada lista cada prueba como `OK` y termina con `10 pruebas pasaron.`

Prueba	Qué verifica	Resultado esperado
`test_fcfs`	Orden por llegada y espera promedio.	Espera 8.75 (0, 7, 10, 18).
`test_sjf_no_apropiativo`	Elección del más corto entre los llegados.	Orden P1, P2, P4, P3; espera 7.75.
`test_srtf`	Expropiación al llegar un proceso más corto.	Segmentos P1, P2, P4, P1, P3; espera 6.5.
`test_round_robin`	Turnos con quantum 4 sobre P1(24), P2(3), P3(3).	Espera promedio 17/3.
`test_round_robin_empate_llegada_y_fin_de_quantum`	Convención de empate en la cola.	El proceso que llega en t = 2 entra antes que el interrumpido.
`test_cpu_ociosa`	Bloques IDLE al inicio y entre llegadas.	`IDLE` en FCFS, SJF, SRTF y Round Robin.
`test_desempate_sjf`	Misma ráfaga: gana la menor llegada.	Orden A, B, C.
`test_prioridad`	Prioridades apropiativo y no apropiativo.	Orden P1, P2, P3 y segmentos con expropiación de P1.
`test_lista_original_no_se_modifica`	Que los algoritmos usen copias.	La lista de entrada queda intacta.
`test_metricas`	Fórmulas de retorno y espera.	P2: `12 - 1 = 11` y `11 - 4 = 7`.

Para añadir una prueba basta una función `test_...` en `tests.py`: el bloque final las descubre y las ejecuta todas.

10) Complejidad, limitaciones y mejoras

Con n procesos, todos los algoritmos son suficientemente rápidos para el tamaño de listas que maneja la interfaz; el costo real lo domina la animación.

Algoritmo	Complejidad (n procesos)	Origen del costo
FCFS	O(n log n)	Ordenar por llegada.
SJF, SRTF, Prioridades	$O(n^2)$	Hasta 2n decisiones (llegadas y términos), cada una recorre los procesos para hallar los listos.
Round Robin	O(n log n + S / q)	S es la suma de las ráfagas y q el quantum: una iteración por turno, cada una de costo constante.

La animación dibuja una unidad por llamada, de modo que su duración crece con el tiempo total de la simulación; el botón Saltar animación evita esa espera.

Limitaciones actuales:

- El tiempo es discreto: llegadas, ráfagas y quantum son enteros.
- No se modelan ráfagas de E/S ni el costo del cambio de contexto.
- La prioridad es estática: sin envejecimiento (aging), un proceso de baja prioridad puede sufrir inanición.
- Solo se usa un quantum para todos los procesos y no hay colas multinivel.
- Con tiempos totales muy largos el Gantt requiere desplazamiento horizontal.

Posibles mejoras:

- Colas multinivel con retroalimentación (MLFQ) y envejecimiento para Prioridades.
- Cambio de contexto con costo configurable.
- Ingreso de ráfagas alternadas de CPU y E/S.
- Exportar el Gantt y las métricas a imagen o CSV.
- Cargar listas de procesos desde un archivo.